

Part 11: Tree Views

Horizontal distance, top, bottom, vertical, boundary, left and right

The idea: give every node a coordinate

So far a node had one coordinate, its level. For view problems we add a second one.

Horizontal distance (HD), sometimes called the column:

$\text{HD}(\text{root}) = 0$

$\text{HD}(\text{left child}) = \text{HD}(\text{parent}) - 1$

$\text{HD}(\text{right child}) = \text{HD}(\text{parent}) + 1$

Level is the vertical coordinate, HD is the horizontal one. Together they place every node on a grid, and every view question becomes a question about that grid.

Our tree with both coordinates:

	hd=-2	hd=-1	hd=0	hd=1	hd=2
level 0			1		
level 1		2		3	
level 2	4		5		6

node :	1	2	3	4	5	6
hd :	0	-1	+1	-2	0	+2
level:	0	1	1	2	2	2

Note that node 1 and node 5 share the column $\text{hd} = 0$. That collision is the interesting part of every view problem.

Once the grid is in your head:

View	Question about the grid
Top view	in each column, the node with the smallest level
Bottom view	in each column, the node with the largest level
Vertical order	in each column, all nodes, top to bottom
Left view	in each row, the leftmost node
Right view	in each row, the rightmost node
Boundary	walk the outline of the drawing

Why BFS is the natural tool

For top view we need “the first node seen in each column”. If you use DFS you might reach column 0 through node 5 at level 2 before ever reaching some other column, and worse, DFS explores the whole left subtree before the right, so it can enter a column at a deep level before a shallower node in the same column is discovered.

BFS visits nodes in increasing order of level, so the **first** time BFS touches a column, that is the topmost node of the column. That single property makes top view a three line change to the BFS template.

If you insist on DFS, you must carry (hd, level) and keep the node with the minimum level per column. That works and is sometimes shorter, but BFS is the version to learn first.

Top view

The nodes you would see looking down at the tree from above.

```
vector<int> topView(TreeNode* root) {
    vector<int> res;
    if (root == NULL) return res;

    map<int,int> col;                                // hd -> value, kept sorted by hd
    queue<pair<TreeNode*,int>> q;                    // node with its hd
    q.push({root, 0});

    while (!q.empty()) {
        auto cur = q.front(); q.pop();
        TreeNode* node = cur.first;
        int hd = cur.second;

        if (col.find(hd) == col.end())                // first node in this column wins
            col[hd] = node->val;

        if (node->left) q.push({node->left, hd - 1});
        if (node->right) q.push({node->right, hd + 1});
    }

    for (auto& p : col) res.push_back(p.second);    // map iterates in increasing hd
    return res;
}
```

Dry run

q = [(1,0)]		
pop (1, 0)	col empty at 0 -> col[0]=1	push (2,-1), (3,1)
pop (2,-1)	col[-1]=2	push (4,-2), (5,0)
pop (3, 1)	col[1]=3	push (6,2)
pop (4,-2)	col[-2]=4	
pop (5, 0)	column 0 already has 1, skip	

```
pop (6, 2)    col[2]=6
```

```
col = { -2:4, -1:2, 0:1, 1:3, 2:6 }
top view = 4 2 1 3 6
```

Node 5 is hidden behind node 1. That is exactly what “looking from above” means.

Time $O(N \log N)$ because of the map. **Space** $O(N)$.

Bottom view

Same code, one word different: always overwrite.

```
col[hd] = node->val;    // last node in this column wins
```

Since BFS processes levels in order, the last node written for a column is the deepest one.

```
col = { -2:4, -1:2, 0:5, 1:3, 2:6 }
bottom view = 4 2 5 3 6
```

Column 0 now shows 5 instead of 1. Everything else is unchanged.

Careful with ties. If two nodes share the same column **and** the same level, which one is “the bottom”? The usual convention, and what BFS gives you naturally, is the one that appears later in left to right order. Different sources define this differently, so if a judge disagrees with you, this is the line to look at.

Vertical order traversal

Now keep everything in each column instead of just one node.

```
vector<vector<int>> verticalOrder(TreeNode* root) {
    vector<vector<int>> res;
    if (root == NULL) return res;

    map<int, vector<int>> col;
    queue<pair<TreeNode*,int>> q;
    q.push({root, 0});

    while (!q.empty()) {
        auto cur = q.front(); q.pop();
        TreeNode* node = cur.first;
        int hd = cur.second;

        col[hd].push_back(node->val);

        if (node->left) q.push({node->left, hd - 1});
        if (node->right) q.push({node->right, hd + 1});
    }
}
```

```

    for (auto& p : col) res.push_back(p.second);
    return res;
}

```

For our tree: `[[4], [2], [1,5], [3], [6]]`.

Time $O(N \log N)$, space $O(N)$.

The LeetCode 987 twist

LeetCode 987 adds a tie-breaking rule: within a column, sort by row first, and if two nodes share the same row and column, sort by **value**. Plain BFS does not guarantee that, so the standard solution collects triples `(col, row, value)`, sorts them, and then groups by column.

```

// collect with DFS or BFS into vector<array<int,3>> v of (col, row, val)
sort(v.begin(), v.end());
// then group consecutive entries with equal col

```

Read the problem statement carefully. LeetCode 314 (vertical order, premium) does not have the value tie-break, LeetCode 987 does. This is a place where copying a remembered solution without reading gets you a wrong answer.

Right view and left view

Covered in Part 3 with BFS: the last node of each level is the right view, the first is the left view. For our tree, right view is 1 3 6 and left view is 1 2 4.

The DFS version is worth knowing because it is shorter:

```

void dfs(TreeNode* root, int level, vector<int>& res) {
    if (root == NULL) return;

    if (level == (int)res.size()) // first node reaching this level
        res.push_back(root->val);

    dfs(root->right, level + 1, res); // right first for the RIGHT view
    dfs(root->left, level + 1, res);
}

```

Visit right before left, and the first node that reaches each new level is the rightmost one. Swap the two calls for the left view. The condition `level == res.size()` is a neat way of asking “is this level new?” without a map.

Boundary traversal

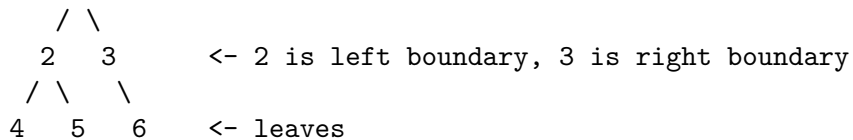
Print the outline of the tree, anticlockwise, starting at the root: left boundary going down, then all leaves left to right, then the right boundary coming up. Each node appears exactly once.

For our tree the answer is 1 2 4 5 6 3.

```

1          <- root

```



Three pieces, and the whole difficulty is avoiding duplicates where they meet.

```

bool isLeaf(TreeNode* n) {
    return n->left == NULL && n->right == NULL;
}

void addLeftBoundary(TreeNode* root, vector<int>& res) {
    TreeNode* cur = root->left;
    while (cur != NULL) {
        if (!isLeaf(cur)) res.push_back(cur->val);    // leaves handled separately
        if (cur->left) cur = cur->left;
        else cur = cur->right;                        // no left child, go right
    }
}

void addLeaves(TreeNode* root, vector<int>& res) {
    if (isLeaf(root)) { res.push_back(root->val); return; }
    if (root->left) addLeaves(root->left, res);
    if (root->right) addLeaves(root->right, res);
}

void addRightBoundary(TreeNode* root, vector<int>& res) {
    TreeNode* cur = root->right;
    vector<int> tmp;
    while (cur != NULL) {
        if (!isLeaf(cur)) tmp.push_back(cur->val);
        if (cur->right) cur = cur->right;
        else cur = cur->left;
    }
    reverse(tmp.begin(), tmp.end());                // bottom to top
    for (int v : tmp) res.push_back(v);
}

vector<int> boundaryTraversal(TreeNode* root) {
    vector<int> res;
    if (root == NULL) return res;

    if (!isLeaf(root)) res.push_back(root->val);    // a single node is a leaf, do not double count

    addLeftBoundary(root, res);
    addLeaves(root, res);
    addRightBoundary(root, res);
    return res;
}

```

The three traps, all about duplicates

1. **The root.** If the tree is a single node, the root is also a leaf. Adding it in both places prints it twice. Hence the `if (!isLeaf(root))` guard.

2. **Leaves on the boundaries.** Node 4 is both on the left boundary and a leaf. The boundary walkers skip leaves, and `addLeaves` handles all of them, in order.
3. **The right boundary direction.** It must come out bottom to top, so collect it into a temporary vector and reverse. Recursion can do this too, by recursing before pushing.

Also note the “if there is no left child, go right” rule inside the boundary walk. The left boundary is not “keep going left”. It is the leftmost path you can trace, which may bend right when it runs out of left children.

Time $O(N)$, space $O(H)$. The leaves walk is $O(N)$, the two boundary walks are $O(H)$ each.

Choosing the data structure

Need	Use	Why
Group by column, output sorted by column	<code>map<int, ...></code>	ordered iteration, no sorting step
Only one node per column	<code>map<int,int></code>	first wins for top, last wins for bottom
All nodes per column	<code>map<int, vector<int>></code>	preserves BFS order inside a column
Node plus its column	<code>queue<pair<TreeNode*,int>></code>	carry the coordinate along with the node
LeetCode 987 tie-breaking	vector of triples plus sort	needs a custom order that a map cannot express
Level based views	plain BFS	no map needed at all

You will see `multimap` mentioned in some references. You rarely need it. `map<int, vector<int>>` is easier to read and does the same job.

If the $O(\log N)$ factor bothers you, note that HD values lie in the range $[-N, N]$, so you can find the minimum and maximum HD in a first pass and use a vector with an offset instead of a map, giving $O(N)$. This is almost never needed on LeetCode, but it is a good answer if an interviewer asks how to remove the log factor.

Common mistakes

- **Using DFS for top or bottom view without tracking the level.** DFS can reach a column at a deep level first and record the wrong node.
- **Using `unordered_map` for columns.** The output must be ordered by column, and `unordered_map` is not. You would have to sort the keys anyway.
- **Updating HD with `hd + 1` on the left child.** Left is minus one. Write the two lines carefully.
- **Forgetting the empty tree.**
- **Boundary traversal printing the root twice** when the root is a leaf.
- **Boundary traversal printing leaves twice**, once as part of a boundary and once from the leaves pass.

- **Not reversing the right boundary.**
- **Believing the left boundary is “always go left”.** When there is no left child you follow the right child.
- **Ignoring LeetCode 987’s value tie-break.**

Check yourself

1. What are the HD values of every node in our sample tree, and which nodes collide?
2. Why does BFS give the top view for free, while DFS needs extra bookkeeping?
3. What is the only change between the top view and the bottom view code?
4. Why is `map` preferred over `unordered_map` here?
5. In boundary traversal, why do the boundary walkers deliberately skip leaves?
6. Why is the right boundary collected and then reversed?
7. Give the top view, bottom view, and boundary traversal of this tree:

